

Laboratorium 8: Analiza struktury i nagłówków plików graficznych

Opracowanie zestawu zadań (część programistyczna)

Kacper Bednarczuk

Zadanie 2.1: Implementacja parsera nagłówków (Wersja Zaawansowana)

Celem zadania było napisanie programu, który bez użycia zewnętrznych bibliotek graficznych (operując jedynie na surowych buforach bajtów) zidentyfikuje typ pliku oraz odczyta jego kluczowe parametry: szerokość, wysokość i głębię kolorów. Zadanie wykonałem w środowisku Node.js.

Program obsługuje formaty **BMP** (Little Endian), **PNG** (Big Endian) oraz dodatkowo **GIF** i **JPEG** (wymagający skanowania markerów).

Kod programu

```
const fs = require("fs");

// Funkcja pomocnicza do analizy pliku BMP (Little Endian)
function analyseBMP(buffer) {
  // BMP przechowuje wymiary na 4 bajtach w Little Endian
  const width = buffer.readUInt32LE(0x12);
  const height = buffer.readUInt32LE(0x16);
  const colourDepth = buffer.readUInt16LE(0x1c);

  console.log("Typ pliku:           BMP");
  console.log(`Szerokość:           ${width} px`);
  console.log(`Wysokość:            ${height} px`);
  console.log(`Głębina kolorów:     ${colourDepth} bitów`);
}

// Funkcja pomocnicza do analizy pliku PNG (Big Endian)
function analysePNG(buffer) {
  // PNG przechowuje wymiary na 4 bajtach w Big Endian w chunku IHDR
  const width = buffer.readUInt32BE(0x10);
  const height = buffer.readUInt32BE(0x14);
  const bitDepth = buffer.readUInt8(0x18);
  const colourType = buffer.readUInt8(0x19);
```

```

    console.log("Typ pliku:          PNG");
    console.log(`Szerokość:        ${width} px`);
    console.log(`Wysokość:         ${height} px`);
    console.log(
        `Głębina bitowa:      ${bitDepth} bitów (Typ koloru: ${colourType})`
    );
}

// Funkcja pomocnicza do analizy pliku GIF (Little Endian)
function analyseGIF(buffer) {
    // GIF przechowuje wymiary w "Logical Screen Descriptor" (zaraz po
    // sygnaturze)
    // Offset 6: Szerokość (2 bajty, LE)
    // Offset 8: Wysokość (2 bajty, LE)
    const width = buffer.readUInt16LE(6);
    const height = buffer.readUInt16LE(8);

    console.log("Typ pliku:          GIF");
    console.log(`Wersja:                ${buffer.toString("ascii", 3, 6)}`); //
    np. 89a
    console.log(`Szerokość:        ${width} px`);
    console.log(`Wysokość:         ${height} px`);
}

// Zaawansowana funkcja do analizy pliku JPEG (skanowanie markerów)
function analyseJPEG(buffer) {
    console.log("Typ pliku:          JPEG");

    // Zaczynamy od offsetu 2 (pomijamy SOI: FF D8)
    let offset = 2;

    while (offset < buffer.length) {
        // Sprawdzamy czy mamy marker (zaczyna się od FF)
        if (buffer[offset] !== 0xff) {
            // Czasami zdarzają się paddingi FF, pomijamy je
            offset++;
            continue;
        }

        const marker = buffer[offset + 1];

        // Szukamy markerów Start Of Frame (SOF0 = C0, SOF2 = C2 dla
        // progressive)
        if (marker === 0xc0 || marker === 0xc2) {
            // Struktura SOF: [len_high, len_low, precision, height_high,
            height_low, width_high, width_low]
            // Offset + 2: Długość segmentu
            // Offset + 4: Precyzja
            // Offset + 5: Wysokość (2 bajty BE)
            // Offset + 7: Szerokość (2 bajty BE)

```

```

    const height = buffer.readUInt16BE(offset + 5);
    const width = buffer.readUInt16BE(offset + 7);

    console.log(`Szerokość:      ${width} px`);
    console.log(`Wysokość:       ${height} px`);
    console.log(
      `Rodzaj:                ${marker === 0xc0 ? "Baseline" : "Progressive"}`
    );
    return; // Znaleźliśmy, kończymy
  }

  // Jeśli to nie SOF, musimy przeskoczyć ten segment
  offset += 2; // Pomijamy bajty markera (FF xx)
  const length = buffer.readUInt16BE(offset); // Czytamy długość segmentu
  offset += length; // Przeskakujemy o długość segmentu
}

console.log("Błąd: Nie znaleziono markera SOF z wymiarami.");
}

function processFile(filePath) {
  console.log(`\n=== Analizuję plik: '${filePath}' ===`);

  try {
    if (!fs.existsSync(filePath)) {
      console.log("Błąd: Plik nie istnieje.");
      return;
    }

    // Wczytujemy nagłówek (64KB powinno wystarczyć nawet dla JPEG)
    const fd = fs.openSync(filePath, "r");
    const buffer = Buffer.alloc(65536);
    const bytesRead = fs.readSync(fd, buffer, 0, 65536, 0);
    fs.closeSync(fd);

    // Identyfikacja po Magicznych Liczbach
    if (buffer[0] === 0x42 && buffer[1] === 0x4d) {
      analyseBMP(buffer);
    } else if (
      buffer[0] === 0x89 &&
      buffer[1] === 0x50 &&
      buffer[2] === 0x4e &&
      buffer[3] === 0x47
    ) {
      analysePNG(buffer);
    } else if (buffer[0] === 0x47 && buffer[1] === 0x49 && buffer[2] ===
0x46) {
      analyseGIF(buffer);
    } else if (buffer[0] === 0xff && buffer[1] === 0xd8) {

```

```
        analyseJPEG(buffer.slice(0, bytesRead));
    } else {
        console.log("Nieznany format pliku lub nieobsługiwana sygnatura.");
        console.log(
            "Nagłówek (HEX):",
            buffer.slice(0, 8).toString("hex").toUpperCase()
        );
    }
} catch (error) {
    console.error("Wystąpił błąd:", error.message);
}
}

// --- TESTY ---
processFile("sample.png");
processFile("sample.bmp");
processFile("sample.gif");
processFile("sample.jpg");
```

Wyniki działania programu

Program poprawnie zidentyfikował wszystkie cztery formaty plików i odczytał ich metadane.

Poniżej przedstawiono logi wygenerowane przez program:

```
=== Analizuję plik: 'sample.png' ===
Typ pliku:          PNG
Szerokość:          50 px
Wysokość:           50 px
Głębina bitowa:     8 bitów (Typ koloru: 6)

=== Analizuję plik: 'sample.bmp' ===
Typ pliku:          BMP
Szerokość:          50 px
Wysokość:           50 px
Głębina kolorów:    24 bitów

=== Analizuję plik: 'sample.gif' ===
Typ pliku:          GIF
Wersja:             89a
Szerokość:          50 px
Wysokość:           50 px

=== Analizuję plik: 'sample.jpg' ===
Typ pliku:          JPEG
Szerokość:          50 px
```

Wysokość:	50 px
Rodzaj:	Baseline

Analiza wyników

1. **Weryfikacja wymiarów:** Dla wszystkich plików (PNG, BMP, GIF, JPEG) program poprawnie odczytał wymiary **50x50 pikseli**. Potwierdza to poprawność implementacji obsługi zarówno formatu **Little-Endian** (używanego w BMP i GIF), jak i **Big-Endian** (używanego w PNG i JPEG).
2. **Szczegóły formatów:**
 - **PNG:** Zidentyfikowano **Typ koloru: 6**. Zgodnie ze specyfikacją PNG, wartość 6 oznacza *Truecolor with Alpha* (RGB + przezroczystość), co zgadza się z właściwościami pliku testowego.
 - **BMP:** Odczytano głębię **24 bity**, co odpowiada standardowemu kodowaniu RGB (8 bitów na kanał).
 - **JPEG:** Parser poprawnie pominął początkowe segmenty pliku (markery) i dotarł do segmentu `S0F0` (Start Of Frame), identyfikując typ kompresji jako **Baseline** (standardowy).

Wniosek: Napisany parser skutecznie automatyzuje proces, który w pierwszej części laboratorium wykonywany był ręcznie przy użyciu edytora szesnastkowego. Wykorzystanie buforów w Node.js pozwala na niskopoziomą analizę plików binarnych bez konieczności stosowania zewnętrznych bibliotek.